Blog Management REST API

Project summary — Node.js, Express, Sequelize, MySQL, JWT · Roles: Admin, User, Guest

1. Overview

A REST API for a blog application with three access levels. It supports registration and login with JWT, user management by admins, blog CRUD with ownership rules, and public blog browsing with title search and category filter. The database is `blogdb` with two tables, `users` and `blogs`.

2. Project Structure

Folder / file	Purpose
`src/server.js`, `src/app.js`	Starts the server, creates the database and tables if missing; sets up Express, CORS, routes, error handlers.
`src/config/`	Environment config (`.env`) and Sequelize/MySQL connection (`ensureDatabase` creates `blogdb`).
`src/models/`	`User` and `Blog` models; `blogs.userId` references `users.id` (ON DELETE CASCADE).
`src/routes/`	`auth`, `users`, `blogs` routes mounted under `/api`.
`src/controllers/`	Business logic for auth, users and blogs.
`src/middleware/`	`authenticate` (JWT), `authorize(role)`, `validate`, `validateId`, 404 and global error handler.
`src/validators/`	Request validation for auth, user and blog payloads, plus shared helpers.
`src/utils/`	`ApiError`, `asyncHandler`, JWT sign/verify helpers.
`scripts/seedAdmin.js`	`npm run seed:admin` creates or promotes an admin user.
`database/schema.sql`	Optional manual SQL for the database and tables.
`postman/`, `README.md`	Postman collection and usage notes.

3. Database

Table	Columns
users	id, firstname, lastname, email (unique), password (bcrypt hash), isActive (default true), role (default user), createAt, updateAt
blogs	id, userId (FK to users.id), blogTitle, blog, category, createAt, updateAt

The `password` column is excluded by default from every query (Sequelize default scope), so it cannot leak into a response by accident.

4. API Endpoints

Method & path	Access	Behaviour
POST `/api/auth/register`	Public	Creates a user with role `user`, isActive true. Any `role` in the body is ignored. 201, or 409 on duplicate email.
POST `/api/auth/login`	Public	Returns a JWT. 401 for invalid credentials, 403 for a deactivated account.
GET `/api/users`	Admin	All users, without passwords.
GET `/api/users/:id`	Admin	One user; 404 if missing, 400 for an invalid id.
PATCH `/api/users/:id/status`	Admin	Activate or deactivate a user (`isActive` boolean). An admin cannot change their own status.
GET `/api/users/profile`	User, Admin	Own profile.
PUT `/api/users/profile/update`	User, Admin	Update firstname / lastname. Sending `role` or `isActive` returns 403.
PATCH `/api/users/password`	User, Admin	Validates, hashes and saves the new password.
GET `/api/blogs`	Public	All blogs with author (id, firstname, lastname). Supports `?title=` (partial match) and `?category=`, alone or together.
GET `/api/blogs/:id`	Public	One blog with author; 404 if missing.
POST `/api/blogs/create`	User, Admin	Creates a blog; `userId` comes from the token, never the body. 201.
PUT `/api/blogs/update/:id`	Owner, Admin	Owner can update own blog; admin can update any. Others get 403.
DELETE `/api/blogs/:id`	Owner, Admin	Same ownership rule (also available at `/api/blogs/delete/:id`).

5. Security and Validation

- Passwords hashed with bcrypt (bcryptjs, 10 rounds); never returned by any endpoint.
- Token-based auth: JWT in the `Authorization: Bearer` header. On every request the user is re-loaded from the database, so deactivated or deleted users are rejected immediately.
- Role checks through `authorize('admin')`; ownership checks in the blog controller.
- Validation: required fields, valid email format, unique email, password minimum 6 characters, non-empty blog title and content, positive-integer IDs, and maximum lengths matching the DB columns.
- LIKE wildcards in title search are escaped, so input such as `50%` is matched literally.
- Error responses are JSON with a `message` (and an `errors` list for validation failures).

Status	Used for
200 / 201	Success / created (register, create blog)
400	Validation failure, invalid ID, malformed JSON
401	Missing or invalid token, wrong credentials
403	Wrong role, not the owner, deactivated account, forbidden profile fields
404	User, blog or route not found
409	Duplicate email

500	Unexpected server error

6. Review and Testing Done

The whole project was checked against the requirement document and then smoke-tested against a running server using a separate throw-away database (`blogdb_test`, dropped afterwards; the real `blogdb` was not touched).

Verified working:

- Registration (201, 409 on duplicate email in any letter case, 400 on bad input); role injection ignored.
- Login for user and admin; wrong password 401; deactivated user 403.
- Admin-only user endpoints (403 for normal users, 404 and 400 cases).
- Deactivation blocks login and invalidates the existing token.
- Profile update restrictions, password change and re-login with the new password.
- Blog create (userId from token), owner vs. other user vs. admin update/delete, guest list with author info, title search, category filter, combined filters, 404 and 400 cases.

> **Bug found and fixed:** a `blogTitle` longer than 255 characters (or a category / name longer than 100) caused a database error and a **500** response. Maximum-length validation was added, so these now return **400** with a clear message. Changed files: `validators/helpers.js` (`MAX_LENGTH`, `isTooLong`), `auth.validator.js`, `user.validator.js`, `blog.validator.js`. The existing structure and style were kept.

7. Notes

- `package.json` lists both `bcrypt` and `bcryptjs`; only `bcryptjs` is used, so `bcrypt` can be removed with `npm uninstall bcrypt`.
- `PUT /users/profile/update` with only `{"role": "admin"}` returns 400 (nothing valid to update); with `firstname` plus `role` it returns 403.
- Set a strong `JWT_SECRET` in `.env` for production; the code falls back to a development default if it is missing.

8. How to Run

- `npm install`, then configure `.env` (see `.env.example`).
- `npm run seed:admin` creates the admin (default `admin@example.com` / `password123`).
- `npm run dev` (or `npm start`) starts the API on the configured port (default 5000).